ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



HỆ ĐIỀU HÀNH

BÁO CÁO ĐỒ ÁN

File & Process Management

Giảng viên hướng dẫn:	Phạm Tuấn Sơn	
	Lê Viết Long	
Sinh viên thực hiện:	Lê Hoàng Hiếu	24120048
	Đặng Hồng Minh	24120090
	Đinh Tiến Phát	24120405

Thành phố Hồ Chí Minh, 11/2026

Mục lục

1 Thông tin nhóm và Phân công công việc **1**

2 Mức độ hoàn thành **1**

3 Tìm hiểu mã nguồn xv6 **2**

3.1 Trường `ofile[]` của `struct proc` và `file[]` của `ftable` . . . 2

3.1.1 Bảng tập tin cục bộ của tiến trình: `ofile[]` . . . 2

3.1.2 Bảng tập tin toàn cục của hệ thống: `ftable` . . . 2

3.1.3 Chi tiết cấu trúc `struct file` . . . 3

3.1.4 Mô hình quan hệ kiến trúc 3 tầng . . . 3

3.2 Trace system call `open` . . . 4

3.2.1 Từ User Space đến Kernel Entry . . . 4

3.2.2 Luồng xử lý chính trong `sys_open()` . . . 4

3.2.3 Các nhánh xử lý lỗi (Error Paths) và Cơ chế dọn dẹp tài nguyên . . . 6

3.3 So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()` . . . 6

3.4 Trace system call `fork` . . . 7

3.5 Các trường `pid`, `state`, `context` của `struct proc` . . . 8

3.5.1 Trường `pid` (Process Identifier) . . . 9

3.5.2 Trường `state` và Vòng đời tiến trình (Process Lifecycle) . . . 9

3.5.3 Trường `context` và Chuyển ngữ cảnh (Context Switching) . . . 10

3.6 Cơ chế lập lịch: `scheduler()`, `sched()`, `yield()` . . . 10

3.6.1 Hàm `scheduler()` – Vòng lặp điều phối của CPU . . . 10

3.6.2 Hàm `sched()` và các điều kiện kiểm tra `panic` . . . 11

3.6.3 Hàm `yield()` và Ngắt thời gian (Timer Interrupt) . . . 12

3.6.4 Cơ chế chuyển giao khóa `p->lock` qua `swtch` . . . 12

4 Cài đặt system call `sysinfo` **13**

4.1 Cấu trúc `struct sysinfo` và luồng gọi . . . 13

4.1.1 Cấu trúc dữ liệu `struct sysinfo` . . . 13

4.1.2 Thách thức về Kích thước Ngăn xếp Kernel và Giải pháp `copyout` từng phần tử . . . 13

4.1.3 Sơ đồ Luồng Gọi Hệ thống `sysinfo` . . . 14

4.2 Thu thập danh sách open file . . . 15

4.2.1 Cài đặt hàm `fileidx()` . . . 15

4.2.2 Cài đặt hàm tra cứu tiến trình `procs_using_file()` . . . 15

4.2.3 Cài đặt hàm thu thập chính `filecollect()` . . . 15

4.2.4 Chiến lược đồng bộ, Xử lý Concurrency và Phòng ngừa Deadlock . . . 16

4.3 Thu thập danh sách process và bộ đếm `nsched` . . . 16

4.3.1 Cài đặt bộ đếm sự kiện lập lịch (`nsched`) . . . 17
4.3.2 Cài đặt hàm thu thập `proccollect()` . . . 17
4.3.3 Các giải pháp kỹ thuật cốt lõi trong `proccollect` . . . 17
4.4 Danh sách các file đã sửa đổi . . . 18

5 Lựa chọn thiết kế và giả định **19**

6 Kết quả kiểm thử **21**
6.1 Cách kiểm thử . . . 21
6.2 Tổng hợp các kịch bản . . . 22
6.3 Một số kết quả tiêu biểu . . . 23

7 Khó khăn và các cách đã thử **25**

8 Những khái niệm chưa hiểu rõ **28**

1 Thông tin nhóm và Phân công công việc

MSSV	Họ và tên	Nhiệm vụ	Mức độ đóng góp
24120048	Lê Hoàng Hiếu	– Tìm hiểu và báo cáo: so sánh cách dùng file descriptor trong `open()`, `dup()`, `fork()`; trace system call `fork`. – Viết chương trình kiểm thử `user/sysinfotest.c`. – Dựng khung báo cáo, viết mục thông tin nhóm, mức độ hoàn thành và kết quả kiểm thử.	100%
24120090	Đặng Hồng Minh	– Tìm hiểu và báo cáo: các trường `pid`, `state`, `context` của `struct proc`; cơ chế lập lịch trong `scheduler()`, `sched()`, `yield()`. – Đăng ký system call `sysinfo` (`syscall.h`, `syscall.c`, `usys.pl`, `user.h`, `Makefile`). – Thêm bộ đếm `nsched` vào `struct proc`, viết `proccollect()` thu thập danh sách process.	100%
24120405	Đinh Tiến Phát	– Tìm hiểu và báo cáo: trường `ofile[]` của `struct proc` và `file[]` của `ftable`; trace system call `open`. – Thiết kế `struct sysinfo` (`kernel/sysinfo.h`), viết `sys_sysinfo()`. – Viết `filecollect()`, `fileidx()`, `procs_using_file()` thu thập danh sách open file.	100%

Mỗi thành viên viết phần báo cáo tương ứng với phần mình tìm hiểu và cài đặt. Các mục 4.4, 5, 7, 8 được ghép từ phần đóng góp của cả ba người.

2 Mức độ hoàn thành

Yêu cầu	Mục	Trạng thái
4.1. Tìm hiểu và báo cáo		
Trường `ofile[]` của `struct proc`, `file[]` của `struct ftable`	3.1	Hoàn thành
Trace system call `open` (luồng chính)	3.2	Hoàn thành
So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()`	3.3	Hoàn thành
Trace system call `fork` (luồng chính)	3.4	Hoàn thành
Các trường `pid`, `state`, `context` của `struct proc`	3.5	Hoàn thành
Cơ chế lập lịch trong `scheduler()`, `sched()`, `yield()`	3.6	Hoàn thành
4.2. Cài đặt và báo cáo		
System call `sysinfo` nhận con trỏ tới `struct sysinfo` (định nghĩa trong `kernel/sysinfo.h`), thông tin được điền trong kernel rồi chép về user space	4.1	Hoàn thành
Số open file của toàn hệ thống và danh sách kèm loại, chế độ mở, reference count, danh sách process tham chiếu tới file	4.2	Hoàn thành
Số process có trạng thái khác `UNUSED` và danh sách kèm pid, trạng thái, pid của cha, số lần được lập lịch, danh sách open file	4.3	Hoàn thành
Chương trình `user/sysinfotest.c` gọi `sysinfo` nhiều lần để kiểm thử nhiều kịch bản (17 kịch bản, 118 kiểm tra, tất cả đều đạt)	6	Hoàn thành

3 Tìm hiểu mã nguồn xv6

3.1 Trường `ofile[]` của `struct proc` và `file[]` của `ftable`

Trong hệ điều hành xv6, việc quản lý tập tin được tổ chức theo kiến trúc phân tầng chặt chẽ nhằm tách biệt không gian định danh cục bộ của từng tiến trình với bảng quản lý tài nguyên tập tin dùng chung trên toàn hệ thống.

3.1.1 Bảng tập tin cục bộ của tiến trình: `ofile[]`

Mỗi tiến trình trong xv6 được đại diện bởi một `struct proc` (được định nghĩa trong `kernel/proc.h`). Bên trong cấu trúc này, mảng `ofile[NOFILE]` gồm các con trỏ `struct file *` quản lý danh sách các tập tin mà tiến trình đang mở.

- **Ý nghĩa của file descriptor (`fd`)**: Chỉ số nguyên $0 \leq$ `fd` $<$ `NOFILE` của mảng chính là giá trị file descriptor mà tiến trình người dùng nhận được khi thực hiện các lời gọi hệ thống như `open()`, `dup()`, hoặc `pipe()`. Theo quy ước POSIX, `fd` $= 0$ là đầu vào chuẩn (standard input), `fd` $= 1$ là đầu ra chuẩn (standard output), và `fd` $= 2$ là đầu ra lỗi chuẩn (standard error).
- **Giới hạn kích thước**: Hằng số `NOFILE` được định nghĩa tại `kernel/param.h` với giá trị bằng 16. Điều này đồng nghĩa với việc mỗi tiến trình chỉ có thể mở đồng thời tối đa 16 tập tin.
- **Cơ chế tham chiếu**: Mỗi phần tử `ofile[fd]` là một con trỏ kiểu `struct file*`. Nếu ô tương ứng chưa được sử dụng hoặc tập tin đã được đóng thông qua `close()`, con trỏ này sẽ mang giá trị `0` (`NULL`). Ngược lại, nó trỏ trực tiếp đến một phần tử nằm trong bảng tập tin toàn cục của hệ thống.

3.1.2 Bảng tập tin toàn cục của hệ thống: `ftable`

Trái ngược với `ofile[]` mang tính cục bộ, `ftable` là bảng quản lý tất cả các đối tượng tập tin đang mở trên toàn bộ hệ điều hành xv6, được khai báo dạng cấu trúc ẩn danh trong `kernel/file.c`, gồm một spinlock `lock` và mảng `file[NFILE]` các `struct file`.

- **Ý nghĩa**: Bảng `ftable.file[]` đóng vai trò là kho lưu trữ tập trung cho mọi tập tin đang mở của hệ thống. Bảng này dùng chung cho tất cả các tiến trình và được bảo vệ bởi khóa `ftable.lock` để đảm bảo an toàn truy cập đồng thời trong môi trường đa nhân (SMP).
- **Giới hạn kích thước**: Hằng số `NFILE` được định nghĩa tại `kernel/param.h` là 100. Do đó, toàn hệ thống xv6 chỉ có thể duy trì tối đa 100 đối tượng tập tin mở đồng thời.
- **Quản lý cấp phát**: Khi một tập tin được mở (thông qua `filealloc()`), kernel duyệt mảng `ftable.file` dưới sự bảo vệ của `ftable.lock` để tìm phần tử có số lượt tham chiếu `ref` $== 0$, tăng `ref` $= 1$ và trả về con trỏ tới phần tử đó. Khi tập tin được đóng (`fileclose()`), `ref` sẽ giảm đi 1; chỉ khi `ref` chạm mốc 0, tập tin mới thực sự bị giải phóng khỏi `ftable`.

3.1.3 Chi tiết cấu trúc `struct file`

Cấu trúc `struct file` (`kernel/file.h`) chứa toàn bộ siêu dữ liệu mô tả trạng thái của một tập tin đang mở, gồm các trường sau:

- `type`: Xác định bản chất tài nguyên mà cấu trúc này đại diện:
  - `FD_NONE`: Ô trống chưa được sử dụng trong `ftable`.
  - `FD_PIPE`: Đường ống giao tiếp liên tiến trình (IPC).
  - `FD_INODE`: Tập tin hoặc thư mục thông thường được lưu trữ trên đĩa.
  - `FD_DEVICE`: Tập tin thiết bị phần cứng đặc biệt (ví dụ: màn hình/bàn phím console).
- `ref`: Số lượng liên kết trỏ tới cấu trúc này (tổng số lượng `ofile[fd]` trên mọi tiến trình đang trỏ đến nó).
- `readable` và `writable`: Các cờ phân quyền truy cập, được thiết lập dựa trên chế độ mở tập tin (`O_RDONLY`, `O_WRONLY`, `O_RDWR`).
- `pipe`: Trỏ tới `struct pipe` chứa bộ đệm vòng (circular buffer) và các khóa đồng bộ khi tập tin là pipe.
- `ip`: Trỏ tới `struct inode` đại diện cho tập tin trên đĩa trong bộ nhớ RAM.
- `off`: Byte offset hiện tại. Mọi thao tác `read()` hoặc `write()` sẽ diễn ra tại vị trí này và tự động tăng giá trị của `off`.
- `major`: Số hiệu thiết bị chính (major number), dùng làm chỉ số tra bảng hàm xử lý `devsw[]`.

3.1.4 Mô hình quan hệ kiến trúc 3 tầng

Kiến trúc xử lý tập tin của xv6 phân chia thành 3 tầng rõ rệt:

1. **Tầng 1 – File Descriptor Cục bộ (`ofile[]`)**: Tồn tại độc lập trong từng tiến trình, giúp che giấu cấu trúc dữ liệu nội bộ của kernel và ngăn chặn tiến trình can thiệp trái phép vào tập tin của tiến trình khác.
2. **Tầng 2 – Bảng Tập tin Mở Toàn cục (`ftable.file[]`)**: Đại diện cho một phiên mở tập tin (open file instance). Đây là nơi lưu trữ quyền truy cập và con trỏ đọc/ghi `off`. Nhờ tầng trung gian này, hai tiến trình (ví dụ cha và con sau `fork()`) hoặc hai descriptor trong cùng một tiến trình (sau `dup()`) có thể chia sẻ chung một con trỏ `off`.
3. **Tầng 3 – Tài nguyên vật lý bên dưới (`inode` / `pipe` / `device`)**: Đại diện cho dữ liệu thực sự trên đĩa cứng hoặc thiết bị ngoại vi. Nếu hai tiến trình thực hiện hai lời gọi `open()` độc lập trên cùng một đường dẫn, chúng sẽ được cấp phát hai `struct file` riêng biệt ở Tầng 2 (với hai con trỏ `off` độc lập), nhưng cả hai `struct file` này đều cùng trỏ về một `struct inode` duy nhất ở Tầng 3.

3.2 Trace system call `open`

Lời gọi hệ thống `open()` là một trong những syscall phức tạp và cốt lõi nhất của hệ thống quản lý tập tin trong xv6. Luồng thực thi chuyển tiếp qua nhiều tầng từ user space, trap handling, VFS cho đến disk log transaction.

3.2.1 Từ User Space đến Kernel Entry

1. **Khởi tạo từ chương trình người dùng**: Chương trình gọi hàm thư viện `open(path, omode)`, ví dụ mở `test.txt` với cờ `O_CREATE | O_RDWR`.

2. **User Trap Stub (`user/usys.S`)**: Đoạn mã hợp ngữ được sinh tự động bởi `usys.pl` nạp số hiệu `SYS_open` (bằng 15, `kernel/syscall.h`) vào thanh ghi `a7` rồi thực hiện lệnh `ecall` để chuyển từ U-mode sang S-mode.

3. **Trampoline và Trap Handling (`kernel/trampoline.S` và `kernel/trap.c`)**:

   - Lệnh `ecall` chuyển CPU sang Supervisor mode, lưu địa chỉ kế tiếp vào thanh ghi `sepc` và nhảy đến địa chỉ vector ngắt tại trang `trampoline`.
   - Hàm `uservec` lưu trữ toàn bộ các thanh ghi người dùng vào trang `p->trapframe`, chuyển con trỏ ngăn xếp sang kernel stack của tiến trình và gọi hàm `usertrap()`.
   - Trong `usertrap()`, kernel kiểm tra nguyên nhân ngắt qua `r_scause() == 8` (mã nguyên nhân gọi syscall từ User mode), tăng `p->trapframe->epc += 4` để khi quay lại user mode sẽ tiếp tục tại lệnh kế tiếp sau `ecall`, sau đó gọi `syscall()`.

4. **Điều phối Syscall (`kernel/syscall.c`)**:

   - Hàm `syscall()` trích xuất số hiệu syscall từ `p->trapframe->a7`.
   - Hệ thống gọi con trỏ hàm tương ứng trong mảng `syscalls[SYS_open]`, kích hoạt hàm `sys_open()` trong `kernel/sysfile.c`.

3.2.2 Luồng xử lý chính trong `sys_open()`

Toàn bộ logic cốt lõi của việc mở tập tin diễn ra trong hàm `sys_open()` (`kernel/sysfile.c:305`):

1. **Lấy tham số từ không gian người dùng**:

   - `argint(1, &omode)`: Lấy giá trị cờ mở tập tin từ thanh ghi `a1`.
   - `argstr(0, path, MAXPATH)`: Đọc chuỗi đường dẫn từ địa chỉ user chỉ định trong thanh ghi `a0` và sao chép vào bộ đệm `path` trong kernel space. Nếu chuỗi không hợp lệ hoặc vượt quá `MAXPATH` (128 bytes), trả về -1.

2. **Bắt đầu giao dịch nhật ký (File System Log Transaction)**:

- Gọi `begin_op()` để bắt đầu một giao dịch nhật ký với hệ thống tập tin, đảm bảo tính nguyên tử (atomicity) khi tạo mới tệp hoặc cắt ngắn kích thước (truncate).

3. **Tìm kiếm hoặc tạo mới Inode**:
   - **Trường hợp có cờ** `O_CREATE`: Gọi hàm `create(path, T_FILE, 0, 0)`. Hàm này sẽ phân giải thư mục cha, tìm kiếm tên tệp trong thư mục. Nếu chưa tồn tại, hàm cấp phát một inode mới từ đĩa (`ialloc()`), tạo mục thư mục liên kết (`dirlink()`) và trả về con trỏ inode đã được khóa (`ilock`).
   - **Trường hợp không có cờ** `O_CREATE`: Gọi `namei(path)` để phân giải đường dẫn thành một con trỏ `struct inode *ip`. Sau đó gọi `ilock(ip)` để đọc thông tin tệp từ đĩa vào RAM và thiết lập khóa loại trừ.
   - **Kiểm tra tệp thư mục**: Nếu `ip->type` == `T_DIR` nhưng chế độ mở `omode` $\neq$ `O_RDONLY`, xv6 từ chối mở vì thư mục chỉ được phép đọc.
4. **Kiểm tra tính hợp lệ của tệp thiết bị**:
   - Nếu `ip->type` == `T_DEVICE`, kernel kiểm tra chỉ số `major` của thiết bị xem có nằm trong khoảng hợp lệ $[0, \texttt{NDEV})$ hay không.
5. **Cấp phát tài nguyên tập tin**:
   - **Cấp phát** `struct file`: Gọi `filealloc()` để tìm một vị trí còn trống (`ref` == 0) trong `ftable.file[]`. Nếu thành công, trường `ref` được gán thành 1.
   - **Cấp phát File Descriptor**: Gọi `fdalloc(f)` để quét mảng `p->ofile[]` từ chỉ số 0 đến `NOFILE` -1. Ô đầu tiên có giá trị `0` sẽ được gán bằng `f`, và chỉ số `fd` này sẽ được giữ lại.
6. **Khởi tạo cấu trúc** `struct file`:
   - Thiết lập kiểu: `f->type` = `FD_DEVICE` (nếu là thiết bị) hoặc `FD_INODE` (nếu là tệp thường).
   - Đặt con trỏ offset ban đầu: `f->off` = 0.
   - Gán con trỏ inode: `f->ip` = `ip`.
   - Thiết lập quyền: `readable` bằng 1 khi `omode` không có `O_WRONLY`; `writable` bằng 1 khi `omode` có `O_WRONLY` hoặc `O_RDWR`.
7. **Xử lý cờ** `O_TRUNC`:
   - Nếu tập tin được mở với cờ `O_TRUNC` và là tập tin thông thường (`T_FILE`), kernel gọi hàm `itrunc(ip)` để giải phóng toàn bộ các khối dữ liệu (data blocks) trên đĩa và đặt lại kích thước tệp về 0.
8. **Mở khóa Inode và Hoàn tất Giao dịch**:
   - Gọi `iunlock(ip)` để giải phóng khóa trên inode (lưu ý inode vẫn được giữ trong bộ nhớ vì con trỏ `f->ip` đang tham chiếu đến nó).

- Gọi `end_op()` để hoàn thành giao dịch nhật ký và ghi dữ liệu ra đĩa nếu cần thiết.
- Trả về số nguyên `fd`. Giá trị này được lưu vào `p->trapframe->a0` để chuyển về cho tiến trình người dùng khi quay lại User mode thông qua `usertrapret()`.

3.2.3 Các nhánh xử lý lỗi (Error Paths) và Cơ chế dọn dẹp tài nguyên

Tính toàn vẹn của hệ điều hành đòi hỏi mọi nhánh lỗi đều phải được xử lý triệt để, không để lại tình trạng rò rỉ bộ nhớ, giữ khóa vô hạn (deadlock) hoặc làm sai lệch hệ thống tập tin:

- **Lỗi phân giải Inode / Tạo tệp thất bại**: Nếu `create()` hoặc `namei()` trả về `0`, kernel lập tức gọi `end_op()` để đóng giao dịch nhật ký và trả về -1.
- **Lỗi kiểm tra quyền hạn hoặc loại tệp (Thư mục mở ghi, Thiết bị không hợp lệ)**: Inode đã được khóa trước đó, do đó kernel phải gọi `iunlockput(ip)` (kết hợp giữa mở khóa `iunlock` và giảm tham chiếu `iput`), sau đó gọi `end_op()` và trả về -1.
- **Lỗi cạn kiệt bảng `ftable` hoặc bảng `ofile[]`**: `sys_open` gọi `filealloc()` rồi mới gọi `fdalloc(f)`, và dùng chung một nhánh lỗi khi một trong hai thất bại. Đây là điểm cần lưu ý đặc biệt: Nếu `filealloc()` thành công (đã tăng `f->ref` $= 1$) nhưng `fdalloc(f)` thất bại (tiến trình đã mở đủ 16 tập tin), kernel phải gọi ngay `fileclose(f)` để hoàn trả ô nhớ trong `ftable`, tránh rò rỉ ô tập tin toàn cục. Tiếp theo, gọi `iunlockput(ip)` và `end_op()` trước khi trả về -1.

3.3 So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()`

Trong xv6, file descriptor (fd) chỉ là chỉ số của mảng `ofile[]` trong `struct proc`. Phần tử ở chỉ số đó trỏ tới một `struct file` trong bảng `ftable` dùng chung cho toàn hệ thống (mục 3.1). Hai trường của `struct file` quyết định các fd có dùng chung trạng thái với nhau hay không: `ref` (số con trỏ trong các `ofile[]` đang trỏ tới nó) và `off` (vị trí đọc/ghi hiện tại). Ba lời gọi `open()`, `dup()`, `fork()` khác nhau ở chỗ có tạo `struct file` mới hay chỉ thêm con trỏ tới một `struct file` đã có.

- `open()` (`sys_open`, `kernel/sysfile.c`): sau khi tìm hoặc tạo được inode, kernel gọi `filealloc()` để lấy ô đầu tiên có `ref == 0` trong `ftable` và đặt `ref = 1`, rồi gọi `fdalloc()` để gắn `struct file` đó vào ô trống có chỉ số nhỏ nhất của `ofile[]`. Quyền đọc/ghi lấy theo `omode`, `off` đặt về 0. Vì mỗi lần `open` tạo một `struct file` riêng nên mở cùng một file hai lần sẽ cho hai fd có offset độc lập; chúng chỉ dùng chung inode. Nếu `fdalloc()` thất bại (process đã dùng hết `NOFILE` fd), `sys_open` gọi `fileclose(f)` để trả lại `struct file` vừa lấy.
- `dup(fd)` (`sys_dup`): `argfd()` kiểm tra fd hợp lệ và lấy `struct file` của nó, `fdalloc()` cấp một fd mới trỏ tới *chính* `struct file` đó, sau đó `filedup()` tăng `ref` thêm 1. Không có `struct file` mới, nên fd cũ và fd mới dùng chung offset và quyền đọc/ghi. `fdalloc()` được gọi trước `filedup()`, nên khi hết fd thì chưa có gì phải hoàn tác.

- `fork()` (`kernel/proc.c`): với mỗi ô khác 0 trong `ofile[]` của cha, kernel gán `np->ofile[i] = filedup(p->ofile[i])`. Con có đúng các số fd như cha, mỗi fd trỏ tới cùng `struct file`, và mỗi fd được chép làm `ref` tăng 1. Cha và con dùng chung offset: con đọc 1 byte thì lần đọc tiếp theo của cha bắt đầu từ byte thứ hai.

Chiều ngược lại, `close()` gọi `fileclose()` để giảm `ref`. Chỉ khi `ref` về 0 thì `struct file` mới được giải phóng (`type = FD_NONE`) và tài nguyên bên dưới mới được trả lại: `iput()` với inode/thiết bị, `pipeclose()` với pipe. Khi process thoát, `exit()` cũng gọi `fileclose()` cho mọi fd còn mở.

	`open()`	`dup()`	`fork()`
Tạo `struct file` mới	Có (`filealloc`)	Không	Không
fd được tạo	1 fd, ô trống nhỏ nhất	1 fd, ô trống nhỏ nhất	Chép mọi fd của cha sang con, giữ nguyên số fd
`ref`	`struct file` mới có `ref = 1`	Tăng 1	Tăng 1 cho mỗi fd được chép
Offset	Riêng, bắt đầu từ 0	Chung với fd gốc	Chung giữa cha và con
Quyền đọc/ghi	Theo `omode`	Giống fd gốc	Giống của cha
Process giữ fd	Process gọi `open`	Process gọi `dup`	Cả cha và con

Bảng 1: So sánh cách dùng file descriptor trong `open()`, `dup()` và `fork()`

Việc `fork()` dùng chung offset là có chủ ý. Shell dựa vào nó để các lệnh con lần lượt ghi tiếp vào cùng một file khi chuyển hướng output, thay vì ghi đè lên nhau [1].

Các tính chất trên được kiểm chứng bằng `sysinfotest` (mục 6). Test 3: hai lần `open` cùng file cho hai entry khác nhau, mỗi entry `ref = 1`, cả hai fd đều đọc ra `'a'`. Test 4: sau `dup` hai fd cùng trỏ một entry có `ref = 2`, đọc lần lượt ra `'a'` rồi `'b'`, và pid chỉ xuất hiện một lần trong danh sách process của file. Test 5: sau `fork` không có entry mới, `ref` của console tăng 3 (con nhận fd 0, 1, 2), con đọc `'a'` thì cha đọc tiếp ra `'b'`. Ngay cả con số `ref = 9` của console lúc đầu cũng giải thích được bằng các quy tắc này: `init` mở console một lần rồi `dup` hai lần (`ref = 3`), sau đó `init` fork ra `sh` và `sh` fork ra `sysinfotest`, mỗi lần fork cộng thêm 3.

3.4 Trace system call `fork`

Luồng chính khi một chương trình user gọi `fork()`:

1. **Vào kernel.** Stub `fork` trong `user/usys.S` nạp `SYS_fork` vào thanh ghi `a7` rồi thực hiện `ecall`. CPU nhảy tới `uservec` (lưu thanh ghi user vào trapframe), rồi `usertrap()` cộng 4 vào `epc` để khi quay về sẽ chạy lệnh ngay sau `ecall`, sau đó `syscall()` gọi `sys_fork()`, và hàm này gọi `fork()` trong `kernel/proc.c`.

2. **`allocproc()`: lấy một ô trong bảng process.** Hàm duyệt `proc[]`, khóa `p->lock` của từng ô để tìm ô `UNUSED`. Khi tìm được, nó cấp pid mới bằng `allocpid()` (tăng `nextpid` dưới `pid_lock`), đặt trạng thái `USED` (và `nsched = 0`, phần nhóm thêm vào), cấp một trang cho trapframe, tạo page table rỗng đã map sẵn trampoline và trapframe (`proc_pagetable()`), xóa `context` rồi đặt `context.ra = forkret` và

`context.sp` bằng đỉnh kernel stack. Hàm trả về khi vẫn giữ `np->lock`, nên chưa CPU nào có thể chạy process mới. Nếu không còn ô trống hoặc hết bộ nhớ, `fork()` trả về -1.

3. `uvmcopy()`**: chép bộ nhớ user.** Với từng trang trong `p->sz` byte của cha, kernel cấp một trang vật lý mới, chép nội dung và map vào page table của con với cùng quyền. xv6 không dùng copy-on-write nên đây là bản sao thật. Nếu lỗi, `freeproc(np)` trả lại mọi thứ đã cấp và `fork()` trả về -1. Sau đó `np->sz = p->sz`.

4. **Chép trapframe.** `*(np->trapframe) = *(p->trapframe)`: con có đúng các thanh ghi user của cha, kể cả `epc`, nên khi về user nó tiếp tục ở cùng chỗ với cha. Riêng `np->trapframe->a0` được gán 0.

5. **Chia sẻ tài nguyên.** `filedup()` cho mọi fd đang mở (mục 3.3), `idup()` cho thư mục hiện hành `cwd`, chép tên process.

6. **Gắn cha và cho phép chạy.** Kernel lưu pid của con, nhả `np->lock`, lấy `wait_lock` để gán `np->parent = p`, rồi lấy lại `np->lock` để đặt trạng thái `RUNNABLE`. Phải nhả `np->lock` trước khi xin `wait_lock` vì thứ tự khóa trong xv6 là `wait_lock` trước `p->lock` (giống `exit()` và `wait()`); giữ `np->lock` rồi mới xin `wait_lock` có thể gây deadlock.

7. **Trả về cho cha.** `fork()` trả về pid của con; `syscall()` ghi giá trị này vào `p->trapframe->a0` của cha, rồi `usertrapret()` đưa cha về user mode.

Lần chạy đầu tiên của con. Khi `scheduler()` gặp con ở trạng thái `RUNNABLE`, nó đặt `RUNNING` (và tăng `nsched`) rồi gọi `swtch(&c->context, &np->context)`. Vì `context.ra = forkret`, lệnh `ret` cuối `swtch` nhảy vào `forkret()` chứ không quay về nơi gọi nào. `forkret()` nhả `p->lock` mà `scheduler()` đang giữ (riêng process đầu tiên của hệ thống còn gọi `fsinit()`), rồi gọi `usertrapret()`; `userret` khôi phục thanh ghi từ trapframe và lệnh `sret` đưa con về user mode tại `epc`.

Vì sao cha nhận pid của con còn con nhận 0. Cả hai quay về user từ cùng một bộ thanh ghi (trapframe của con là bản sao), chỉ khác thanh ghi `a0` là nơi chứa giá trị trả về: `a0` của cha được `syscall()` ghi bằng giá trị trả về của `fork()` (pid của con), còn `a0` của con đã bị `fork()` gán 0 từ trước và con không hề đi qua đoạn ghi giá trị trả về trong `syscall()`. Chương trình user phân biệt cha và con dựa vào giá trị này.

Kết quả kiểm thử khớp với luồng trên. Test 5: ngay sau `fork`, con có `ppid` bằng pid của cha, cùng tên `sysinfotest`, cùng các fd, ở trạng thái `RUNNABLE` với `nsched = 0` vì chưa được chọn chạy lần nào. Test 16: fork liên tục cho tới khi `allocproc()` hết ô, tạo được 61 con, `nproc` đạt `NPROC` $= 64$ và lần `fork` tiếp theo trả về -1. Test 13: khi cha thoát, `exit()` gọi `reparent()` chuyển con của nó cho `init` (`ppid` thành 1).

3.5 Các trường `pid`, `state`, `context` của `struct proc`

Trong hệ điều hành xv6, mỗi tiến trình được đại diện bởi một cấu trúc `struct proc` (`kernel/proc.h`). Bảng tiến trình toàn cục `proc[NPROC]` gồm tối đa 64 phần tử (`NPROC = 64`). Dưới đây là phân tích chi tiết về 3 trường quan trọng liên quan đến định danh, trạng thái vòng đời và chuyển ngữ cảnh:

3.5.1 Trường `pid` (Process Identifier)

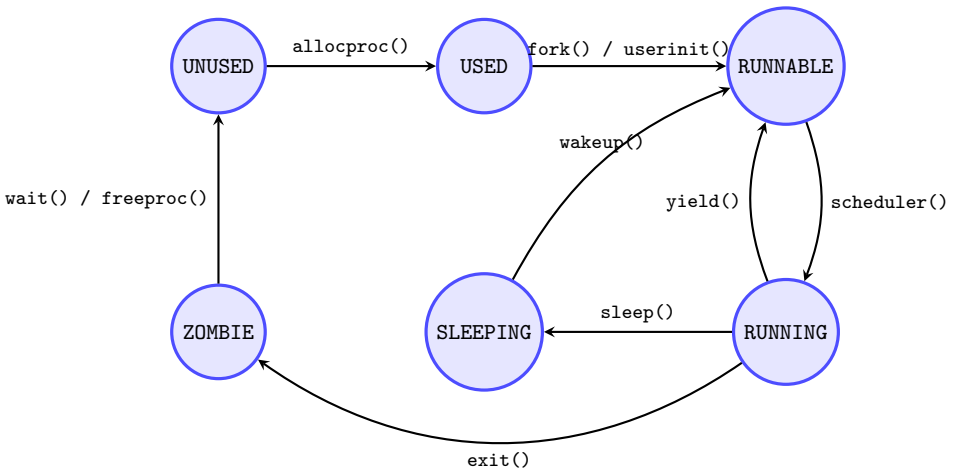
- **Cơ chế cấp phát:** Trường `pid` được cấp phát trong hàm `allocproc()` (`kernel/proc.c`) thông qua hàm trợ giúp `allocpid()`. Biến toàn cục `nextpid` được bảo vệ bởi khóa `pid_lock`:
  - Khi một tiến trình được tạo mới, hàm lấy khóa `acquire(&pid_lock)`, tăng `nextpid` lên 1 đơn vị và gán giá trị này cho tiến trình mới, sau đó giải phóng khóa.
  - Cơ chế này đảm bảo tính đơn điệu tăng dần của PID trên hệ thống đa lõi (multi-core SMP).
- **Không tái sử dụng PID:** Trong xv6, `nextpid` chỉ tăng dần mà không quay vòng hay tái sử dụng lại các PID của tiến trình đã kết thúc. Điều này giúp loại trừ hoàn toàn nguy cơ tranh chấp race condition khi một tiến trình gửi tín hiệu (ví dụ `kill()`) đến một PID cũ vừa bị tái sử dụng cho tiến trình khác.

3.5.2 Trường `state` và Vòng đời tiến trình (Process Lifecycle)

Trường `state` có kiểu `enum procstate` gồm 6 trạng thái cơ bản:

Trạng thái	Ý nghĩa
`UNUSED`	Slot trong bảng `proc[]` đang trống, chưa có tiến trình nào chiếm giữ.
`USED`	Slot vừa được `allocproc()` cấp phát và đang trong quá trình thiết lập.
`RUNNABLE`	Tiến trình đã sẵn sàng để thực thi, đang nằm trong hàng đợi chờ CPU.
`RUNNING`	Tiến trình đang thực sự được một lõi CPU thực thi lệnh.
`SLEEPING`	Tiến trình đang bị chặn, chờ đợi một sự kiện (kênh `chan`) qua `sleep()`.
`ZOMBIE`	Tiến trình đã gọi `exit()` nhưng cha của nó chưa gọi `wait()` để thu gom.

3.5.2.1 Sơ đồ chuyển trạng thái:



3.5.2.2 Quy tắc đồng bộ khóa `p->lock`: Bất kỳ thao tác đọc hoặc ghi nào lên trường `state` đều bắt buộc phải giữ spinlock `p->lock`. Lý do:

- Trên hệ thống đa CPU, hàm `scheduler()` trên từng CPU liên tục duyệt qua bảng `proc[]` để tìm tiến trình ở trạng thái `RUNNABLE`. Nếu không có `p->lock`, hai CPU có thể cùng lúc thấy một tiến trình là `RUNNABLE` và cùng chọn nó vào chạy, dẫn đến xung đột bộ nhớ nghiêm trọng.
- Việc chuyển trạng thái sang `SLEEPING` hoặc `ZOMBIE` đi kèm với các thao tác chuyển luồng điều khiển; khóa `p->lock` bảo vệ tính nguyên tử của cả chuỗi thao tác này.

3.5.3 Trường `context` và Chuyển ngữ cảnh (Context Switching)

Cấu trúc `struct context` (`kernel/proc.h`) chứa 14 trường thanh ghi 64-bit của kiến trúc RISC-V:

- `ra`: Return Address (địa chỉ lệnh sẽ quay về khi kết thúc hàm).
- `sp`: Stack Pointer (con trỏ đỉnh ngăn xếp của kernel thread).
- `s0` – `s11`: Callee-saved registers (các thanh ghi mà hàm được gọi có nghĩa vụ bảo toàn giá trị cho hàm gọi nó theo quy ước gọi hàm của RISC-V ABI).

3.5.3.1 Phân biệt `struct context` và `struct trapframe`:

Tiêu chí	`struct context`	`struct trapframe`
Không gian lưu	Kernel context	User context
Mục đích	Chuyển đổi giữa 2 **kernel thread** (giữa tiến trình và vòng lặp `scheduler()`).	Lưu trữ trạng thái của **chương trình người dùng** khi xảy ra ngắt/ngoại lệ/syscall.
Tập thanh ghi lưu	Chỉ lưu các thanh ghi **callee-saved** (`ra`, `sp`, `s0`–`s11`). Các thanh ghi caller-saved tự động được lưu trên kernel stack nhờ trình biên dịch C.	Lưu **toàn bộ thanh ghi người dùng** (32 thanh ghi RISC-V) vì tiến trình user có thể bị ngắt tại bất kỳ lệnh assembly nào.
Cơ chế thực hiện	Hàm hợp ngữ `swtch()` (`kernel/swtch.S`) được gọi tường minh từ mã C trong kernel.	Mã hợp ngữ `uservec` / `userret` trong `kernel/trampoline.S` khi chuyển chế độ U-mode ↔ S-mode.
Vị trí lưu trữ	Nằm trực tiếp bên trong `struct proc` của tiến trình hoặc `struct cpu` của CPU.	Một trang bộ nhớ riêng (`PGSIZE = 4KB`) ánh xạ ngay dưới trang trampoline trong không gian địa chỉ tiến trình.

Bảng 2: So sánh giữa `struct context` và `struct trapframe`

3.6 Cơ chế lập lịch: `scheduler()`, `sched()`, `yield()`

Hệ điều hành xv6 triển khai giải thuật lập lịch luân phiên Round-Robin trên từng lõi CPU. Mỗi lõi CPU sở hữu một kernel thread riêng biệt thực thi hàm `scheduler()` (`kernel/proc.c`), độc lập hoàn toàn với các luồng kernel của các tiến trình người dùng.

3.6.1 Hàm `scheduler()` – Vòng lặp điều phối của CPU

Mỗi CPU sau khi hoàn tất khởi tạo ở `main.c` sẽ nhảy vào `scheduler()` và không bao giờ thoát ra (`__attribute__((noreturn))`):

1. CPU bật ngắt (`intr_on()`) ở đầu vòng lặp để tránh deadlock trong trường hợp tất cả tiến trình đều đang chờ ngắt thiết bị I/O.
2. CPU duyệt qua bảng `proc[0 ...NPROC-1]`. Với mỗi tiến trình, CPU lấy khóa `acquire(&p->lock)`.
3. Nếu phát hiện `p->state == RUNNABLE`:
   - Đổi trạng thái tiến trình sang thực thi: `p->state = RUNNING`.
   - Ghi nhận tiến trình đang chạy trên CPU hiện tại: `c->proc = p`.
   - Tăng bộ đếm số lần được lập lịch: `p->nsched++`.
   - Thực hiện chuyển ngữ cảnh bằng hàm `swtch(&c->context, &p->context)`. Tại thời điểm này, CPU lưu trữ thanh ghi hiện tại của scheduler vào `c->context` và nạp thanh ghi của `p->context` để bắt đầu chạy luồng của tiến trình.
   - Khi tiến trình từ bỏ CPU và quay lại scheduler (sau khi kết thúc `swtch` từ phía tiến trình), scheduler đặt lại `c->proc = 0`.
4. CPU giải phóng khóa `release(&p->lock)` và tiếp tục vòng lặp kiểm tra tiến trình tiếp theo.
5. Nếu duyệt hết bảng mà không có tiến trình nào khả dụng (`found == 0`), CPU thực hiện lệnh hợp ngữ `wfi` (Wait For Interrupt) để tạm dừng tiêu thụ điện năng cho tới khi có ngắt phần cứng xuất hiện.

3.6.2 Hàm `sched()` và các điều kiện kiểm tra `panic`

Hàm `sched()` là cổng duy nhất để một tiến trình kernel trả quyền điều khiển về cho luồng `scheduler()` của CPU. Trước khi chuyển ngữ cảnh, hàm kiểm tra bốn điều kiện dưới đây, vi phạm điều nào cũng gọi `panic`. Sau đó hàm lưu `mycpu()->intena`, gọi `swtch(&p->context, &mycpu()->context)` để chuyển về luồng scheduler, và khôi phục `intena` khi tiến trình được chạy lại.

3.6.2.1 Ý nghĩa của từng điều kiện `panic`:

- `!holding(&p->lock)`: Bắt buộc phải giữ khóa `p->lock`. Khóa này bảo vệ trạng thái của tiến trình khỏi các CPU khác khi tiến trình chuyển sang `RUNNABLE`, `SLEEPING` hoặc `ZOMBIE`. Nếu không giữ khóa, một CPU khác có thể lập tức phát hiện và chạy tiến trình trước khi lệnh `swtch()` kịp hoàn tất việc lưu ngữ cảnh.
- `mycpu()->noff != 1`: Tiến trình chỉ được phép giữ **duy nhất 1 khóa** là `p->lock` (`noff` là độ sâu lồng nhau của các lệnh `push_off()`, tức số lượng spinlock đang nắm giữ). Nếu tiến trình đang giữ thêm khóa khác (ví dụ khóa đĩa, khóa file, `wait_lock`) mà nhường CPU, các tiến trình khác cần khóa đó sẽ bị nghẽn (spinning) vô hạn, dẫn đến nguy cơ deadlock toàn hệ thống.
- `p->state == RUNNING`: Trước khi gọi `sched()`, tiến trình bắt buộc phải chuyển trạng thái sang một giá trị khác (`RUNNABLE` trong `yield()`, `SLEEPING` trong `sleep()`, hoặc `ZOMBIE` trong `exit()`). Nếu vẫn giữ trạng thái `RUNNING`, việc nhường CPU là vô nghĩa và biểu hiện lỗi luận lý trong kernel.

- `intr_get()`: Ngắt phần cứng trên CPU phải đang bị tắt. Điều này được đảm bảo tự động bởi cơ chế `push_off()` khi lấy `p->lock`. Việc tắt ngắt giúp đảm bảo chuỗi thao tác lưu ngữ cảnh và chuyển luồng diễn ra nguyên tử, không bị gián đoạn giữa chừng bởi ngắt timer hay ngắt ngoại vi.

3.6.3 Hàm `yield()` và Ngắt thời gian (Timer Interrupt)

- **Cơ chế hoạt động:** `yield()` lấy `p->lock`, đặt `p->state = RUNNABLE` rồi gọi `sched()` để nhường CPU. Khi tiến trình được chọn chạy lại, `sched()` trả về và `yield()` nhả `p->lock`.
- **Kích hoạt:** Hàm `yield()` được gọi tự động khi hết quantum thời gian. Phần cứng bộ đếm thời gian (CLINT/ACLINT timer) định kỳ kích hoạt ngắt phần cứng. Tại `usertrap()` hoặc `kerneltrap()` (`kernel/trap.c`), nếu `which_dev == 2` (ngắt timer) thì kernel gọi `yield()`. Nhờ cơ chế này, một tiến trình chạy vòng lặp vô hạn ở user space cũng không thể độc chiếm CPU; hệ điều hành có thể cướp quyền điều khiển (preemptive multitasking) để nhường lượt cho các tiến trình khác.

3.6.4 Cơ chế chuyển giao khóa `p->lock` qua `swtch`

Một trong những điểm tinh vi nhất trong thiết kế đồng bộ của xv6 là cơ chế **chuyển giao khóa bất đối xứng qua ranh giới luồng thực thi**:

- Thông thường trong lập trình đồng thời, một khóa được `acquire()` và `release()` trong cùng một hàm và cùng một luồng. Tuy nhiên đối với `p->lock`:
  - **Chiều từ Scheduler sang Process:** Luồng `scheduler()` gọi `acquire(&p->lock)`, thiết lập `p->state = RUNNING`, rồi gọi `swtch(&c->context, &p->context)`. Lúc này luồng của tiến trình `p` tiếp tục chạy từ vị trí nó đã dừng trước đó (hoặc từ `forkret()`). Chính **luồng của tiến trình** `p` sẽ chịu trách nhiệm gọi `release(&p->lock)` (ví dụ trong `yield()` hoặc `forkret()`).
  - **Chiều từ Process sang Scheduler:** Khi tiến trình muốn nhường CPU (trong `yield()`, `sleep()` hay `exit()`), chính **tiến trình** `p` gọi `acquire(&p->lock)`, đổi trạng thái, rồi gọi `swtch(&p->context, &c->context)`. Ngay sau khi `swtch` hoàn tất việc chuyển giao trở lại luồng scheduler, chính **luồng scheduler** sẽ thực hiện `release(&p->lock)`.
- **Mục đích kiến trúc:** Cơ chế này đảm bảo rằng trạng thái của tiến trình `p` luôn được bảo vệ liên tục, xuyên suốt quá trình chuyển giao ngữ cảnh giữa hai lõi CPU khác nhau, ngăn chặn hoàn toàn tình huống một CPU khác can thiệp vào tiến trình khi nó đang trong quá trình chuyển đổi dở dang.

4 Cài đặt system call `sysinfo`

4.1 Cấu trúc `struct sysinfo` và luồng gọi

4.1.1 Cấu trúc dữ liệu `struct sysinfo`

Để cung cấp bức tranh toàn diện và chính xác về trạng thái quản lý tập tin và tiến trình trong hệ điều hành xv6, cấu trúc `struct sysinfo` cùng các cấu trúc con được định nghĩa trong `kernel/sysinfo.h` theo chuẩn thiết kế đã thống nhất giữa các thành viên (Bảng 3). Process và file được nối với nhau qua chỉ số trong `ftable`: fd thứ k của một process trỏ tới file có `idx` bằng `fidx[`k`]`.

Trường	**Ý nghĩa**
`struct sysinfo`	
`nfile`, `files[NFILE]`	Số file đang mở trong toàn hệ thống và danh sách của chúng
`nproc`, `procs[NPROC]`	Số process có trạng thái khác `UNUSED` và danh sách của chúng
`struct sysinfo_file`: một file đang mở	
`idx`	Vị trí trong `ftable.file[]`, dùng để nối với process
`type`	Loại file: `FD_PIPE`, `FD_INODE` hoặc `FD_DEVICE`
`readable`, `writable`	Chế độ mở
`ref`	Reference count
`npids`, `pids[NPROC]`	Các process có ít nhất một fd trỏ tới file này
`struct sysinfo_proc`: một process	
`pid`, `state`	pid và trạng thái (giá trị của `enum procstate`)
`ppid`	pid của cha, bằng 0 nếu không có cha (`init`)
`nsched`	Số lần được scheduler chọn chạy
`name[16]`	Tên process
`nfd`, `fds[NOFILE]`, `fidx[NOFILE]`	Số fd đang mở, các số fd, và `idx` của file mà mỗi fd trỏ tới

Bảng 3: Các trường của `struct sysinfo`

4.1.2 Thách thức về Kích thước Ngăn xếp Kernel và Giải pháp `copyout` từng phần tử

Vấn đề tràn ngăn xếp (Stack Overflow) trong Kernel:

- Kích thước tính toán của các cấu trúc:

$$\text{sizeof}(\texttt{struct sysinfo_file}) = 4 \times 5 + 4 + 4 \times 64 = 280 \text{ bytes}$$

$$\text{sizeof}(\texttt{files}[100]) = 280 \times 100 = 28,000 \text{ bytes} \approx 28 \text{ KB}$$

$$\text{sizeof}(\texttt{struct sysinfo_proc}) = 4 \times 4 + 16 + 4 + 16 \times 4 + 16 \times 4 = 164 \text{ bytes}$$

$$\text{sizeof}(\texttt{procs}[64]) = 164 \times 64 = 10,496 \text{ bytes} \approx 10.5 \text{ KB}$$

$$\text{Tổng kích thước } \texttt{struct sysinfo} \approx 28 \text{ KB} + 10.5 \text{ KB} + 8 \text{ bytes} \approx 38.5 \text{ KB}$$

- Trong xv6, ngăn xếp chế độ nhân (kernel stack) của mỗi tiến trình chỉ được cấp phát đúng **1 trang bộ**

nhớ vật lý duy nhất (`PGSIZE = 4096 bytes = 4 KB`), và ngăn xếp của tiến trình người dùng cũng chỉ có 1 trang (`USERSTACK = 1` trong `kernel/param.h`).

- Nếu bất kỳ hàm nào trong kernel khai báo một biến cục bộ dạng `struct sysinfo info;` trên ngăn xếp, thao tác này sẽ tiêu tốn ngay lập tức $\approx$ 38.5 KB, vượt gấp gần 10 lần dung lượng ngăn xếp cho phép (4 KB). Hậu quả là ngăn xếp lập tức bị tràn qua trang bảo vệ (guard page), làm sập hệ điều hành (kernel panic do page fault).

Giải pháp Kiến trúc:

- Trong kernel, tuyệt đối **không** khai báo toàn bộ cấu trúc `struct sysinfo` trên ngăn xếp.
- Thay vào đó, kernel chỉ khai báo **duy nhất một phần tử** cục bộ (`struct sysinfo_file sf` $\approx$ 280 bytes trong `filecollect`, hoặc `struct sysinfo_proc sp` $\approx$ 164 bytes trong `proccollect`) trên stack tại mỗi vòng lặp.
- Ngay sau khi trích xuất và chuẩn bị xong dữ liệu cho phần tử thứ k, kernel sử dụng hàm `copyout()` để sao chép trực tiếp phần tử đó vào đúng vị trí đích trên không gian địa chỉ ảo của tiến trình người dùng. Với mảng `files`, vị trí đó là `addr` cộng offset của trường `files` trong `struct sysinfo` (tính bằng `__builtin_offsetof`) cộng $k\times$ `sizeof(struct sysinfo_file)`; mảng `procs` tính tương tự.
- Ở phía chương trình người dùng (`sysinfotest.c`), do ngăn xếp user cũng chỉ có 1 trang (4KB), biến `struct sysinfo` bắt buộc phải được khai báo ở phạm vi **toàn cục** (được lưu tại phân vùng BSS/DATA) hoặc cấp phát động qua `malloc()`, không được để làm biến cục bộ trong hàm.

4.1.3 Sơ đồ Luồng Gọi Hệ thống `sysinfo`

Quy trình kích hoạt và điều phối của syscall `sysinfo` được tóm lược như sau:

1. **User Space**: Tiến trình người dùng gọi `sysinfo(&info)`.
2. **Traps**: Lệnh `ecall` đưa CPU vào kernel $\rightarrow$ `usertrap()` $\rightarrow$ `syscall()` phân phối tới `sys_sysinfo()` (`kernel/sysproc.c`).
3. **Điều phối trong** `sys_sysinfo()`:
   - Gọi `argaddr(0, &addr)` để lấy địa chỉ con trỏ buffer người dùng.
   - Gọi `nfile = filecollect(addr)` (`kernel/file.c`): duyệt bảng `ftable`, thu thập các file đang mở và ghi trực tiếp vào mảng `info.files` của user.
   - Gọi `nproc = proccollect(addr)` (`kernel/proc.c`): duyệt mảng `proc`, thu thập các tiến trình đang hoạt động và ghi trực tiếp vào mảng `info.procs` của user.
   - Thực hiện `copyout()` hai biến đếm `nfile` và `nproc` vào các trường đầu của cấu trúc tại địa chỉ `addr`.
   - Trả về 0 nếu toàn bộ thành công, hoặc -1 nếu có bất kỳ lỗi sao chép bộ nhớ nào (địa chỉ user không hợp lệ).

4.2 Thu thập danh sách open file

Phần thu thập thông tin về các tập tin đang mở trên hệ thống đòi hỏi sự kết hợp chặt chẽ giữa việc trích xuất dữ liệu từ bảng `ftable` và tra cứu ngược danh sách các tiến trình đang sở hữu file descriptor trỏ tới từng tập tin.

4.2.1 Cài đặt hàm `fileidx()`

Hàm `fileidx()` được bổ sung vào `kernel/file.c` nhằm ánh xạ từ con trỏ `struct file *f` sang chỉ số nguyên trong bảng `ftable.file[]`: hàm trả về `f - ftable.file`, hoặc -1 nếu `f` không nằm trong bảng.

Hàm này cho phép thành viên phụ trách phần Process (thành viên C) khi duyệt qua mảng `ofile[]` của tiến trình có thể dễ dàng chuyển đổi con trỏ tập tin thành một chỉ số định danh thống nhất (`fidx`), từ đó liên kết chính xác giữa tiến trình và tập tin tương ứng trong báo cáo `sysinfo`.

4.2.2 Cài đặt hàm tra cứu tiến trình `procs_using_file()`

Để xác định những tiến trình nào đang mở một tập tin `f`, hàm `procs_using_file(f, pids)` được cài đặt trong `kernel/proc.c` (nơi nắm giữ mảng toàn cục `proc[NPROC]`). Hàm duyệt `proc[]`; với mỗi tiến trình khác `UNUSED`, hàm giữ `p->lock` trong lúc quét `ofile[]` tìm ô trỏ tới `f`, nhả khóa rồi mới ghi pid vào mảng `pids`. Hàm trả về số pid tìm được.

Các điểm kỹ thuật quan trọng:

- **Khóa bảo vệ từng tiến trình**: Với mỗi tiến trình `p`, hàm lấy khóa `acquire(&p->lock)` để đảm bảo trạng thái `p->state` và mảng `p->ofile` được kiểm tra một cách nhất quán, tránh đọc phải tiến trình đang trong trạng thái khởi tạo dở dang hoặc đang giải phóng.
- **Loại trừ PID trùng lặp**: Sau khi thực hiện lệnh `dup()` hoặc mở cùng một file trên nhiều descriptor khác nhau trong cùng một tiến trình, `p->ofile[]` có thể chứa nhiều ô cùng trỏ tới con trỏ `f`. Vòng lặp kiểm tra sử dụng cờ `found` và lệnh `break` ngay khi tìm thấy descriptor đầu tiên, đảm bảo mỗi PID chỉ được đưa vào danh sách `pids[]` đúng một lần duy nhất.
- **An toàn bộ đệm**: Do hệ thống xv6 chỉ có tối đa `NPROC` tiến trình và mỗi tiến trình chỉ được thêm tối đa một lần, số lượng phần tử `npids` luôn thỏa mãn $0 \leq$ `npids` $\leq$ `NPROC`, hoàn toàn không có nguy cơ tràn mảng `pids[NPROC]`.

4.2.3 Cài đặt hàm thu thập chính `filecollect()`

Hàm `filecollect(addr)` trong `kernel/file.c` duyệt qua toàn bộ bảng `ftable` và sao chép thông tin từng tập tin vào không gian người dùng. Với mỗi ô thứ i của `ftable.file[]`:

1. Lấy `ftable.lock`. Nếu ô có `ref == 0` thì nhả khóa và bỏ qua ô này.
2. Xóa biến cục bộ `sf` bằng `memset`, chép `idx = i`, `type`, `readable`, `writable`, `ref` vào `sf`, rồi nhả `ftable.lock`.

3. Gọi `procs_using_file()` để điền `npids` và `pids` (lúc này không còn giữ `ftable.lock`).

4. `copyout` biến `sf` vào phần tử thứ `count` của mảng `files` ở phía user rồi tăng `count`; nếu `copyout` lỗi thì trả về -1.

Cuối cùng hàm trả về `count`, chính là số file đang mở.

4.2.4 Chiến lược đồng bộ, Xử lý Concurrency và Phòng ngừa Deadlock

Trong quá trình thiết kế và cài đặt, hai vấn đề đồng bộ cốt lõi đã được giải quyết một cách triệt để:

1. **Tuyệt đối không gọi `copyout()` khi đang giữ Spinlock**: Hàm `copyout()` có thể tốn nhiều chu kỳ máy để dịch địa chỉ ảo sang địa chỉ vật lý và sao chép bộ nhớ giữa các trang. Giữ một spinlock (như `ftable.lock`) trong suốt quá trình này sẽ làm vô hiệu hóa ngắt trên CPU cục bộ trong thời gian dài và khiến các CPU khác bị nghẽn (spinning) lãng phí. Do đó, khóa `ftable.lock` luôn được giải phóng trước khi gọi `copyout()`.

2. **Phòng tránh Deadlock giữa `ftable.lock` và `p->lock`**:

   - Trong thiết kế ban đầu của xv6, không có một quy ước thứ tự phân cấp (lock ordering hierarchy) định trước giữa khóa bảng tập tin `ftable.lock` và khóa tiến trình `p->lock`.
   - Nếu trong hàm `filecollect()`, kernel giữ `ftable.lock` rồi gọi `procs_using_file()` (hàm này lần lượt acquire `p->lock`), đồng thời trên một CPU khác có một tiến trình đang gọi `sys_open()` hoặc `sys_close()` (tiến trình này đang giữ `p->lock` và cố gắng acquire `ftable.lock`), hệ thống sẽ lập tức rơi vào **vòng tròn chờ đợi (circular wait)** và gây ra **Deadlock** treo cứng toàn bộ hệ thống.
   - **Kỹ thuật Snapshot giải quyết**: Kernel áp dụng kỹ thuật chụp trạng thái (snapshot): Khi duyệt tới một file hợp lệ, kernel lấy `ftable.lock`, sao chép nhanh các thuộc tính cơ bản (`type`, `readable`, `writable`, `ref`) vào biến cục bộ `sf` trên stack, và **lập tức giải phóng `ftable.lock`**. Sau khi đã nhả khóa, hàm mới an toàn gọi `procs_using_file()` để tra cứu các tiến trình sở hữu.

3. **Tính nhất quán của dữ liệu thống kê (Statistical Snapshot)**: Việc giải phóng `ftable.lock` trước khi tra cứu tiến trình đồng nghĩa với việc có một cửa sổ thời gian rất nhỏ (race window) mà một tiến trình khác có thể vừa đóng (`close`) tập tin. Tuy nhiên, đối với một system call mang tính chất điều tra, quan sát và thống kê hệ thống như `sysinfo`, việc chấp nhận dữ liệu phản ánh trạng thái nhất quán tại thời điểm lấy mẫu (snapshot at sampling time) là hoàn toàn hợp lý, chuẩn mực trong thiết kế hệ điều hành, vừa bảo đảm an toàn hệ thống, vừa giữ được hiệu năng cao nhất.

4.3 Thu thập danh sách process và bộ đếm `nsched`

Để cung cấp thông tin chi tiết về các tiến trình đang hoạt động trong hệ thống theo yêu cầu của system call `sysinfo`, thành viên C chịu trách nhiệm cài đặt bộ đếm lập lịch `nsched` và hàm thu thập danh sách tiến trình `proccollect()` trong `kernel/proc.c`.

4.3.1 Cài đặt bộ đếm sự kiện lập lịch (nsched)

- **Khai báo trường:** Trường `int nsched;` được bổ sung vào `struct proc` (`kernel/proc.h`) trong nhóm các trường dữ liệu được bảo vệ bởi khóa `p->lock`.
- **Khởi tạo:** Trong hàm `allocproc()` (`kernel/proc.c`), ngay sau khi slot tiến trình được cấp pid và gán trạng thái `USED`, `p->nsched` được khởi tạo về 0.
- **Tăng giá trị:** Trong hàm `scheduler()` (`kernel/proc.c`), khi chọn được một tiến trình `RUNNABLE`, sau khi đặt trạng thái `RUNNING` và `c->proc = p`, ta tăng `p->nsched` ngay trước lệnh `swtch(&c->context, &p->context)`.
- **Tính an toàn đồng bộ:** Thao tác tăng `p->nsched++` diễn ra khi CPU đang nắm giữ `p->lock`, do đó không bao giờ xảy ra race condition giữa các CPU khi cùng quan sát tiến trình.

4.3.2 Cài đặt hàm thu thập `proccollect()`

Hàm `int proccollect(uint64 addr)` có nhiệm vụ duyệt qua toàn bộ bảng `proc[0 ...NPROC-1]`, thu thập các tiến trình có trạng thái khác `UNUSED`, và ghi dữ liệu ra không gian người dùng tại địa chỉ `addr`. Với mỗi ô `p` của `proc[]`:

1. Lấy `wait_lock` rồi `p->lock` (đúng thứ tự khóa của xv6, xem mục 4.3.3). Nếu `p` là `UNUSED` thì nhả hai khóa và bỏ qua.
2. Xóa biến cục bộ `sproc` bằng `memset`, chép `pid`, `state`, `nsched`, `name`; `ppid` lấy từ `p->parent->pid`, hoặc bằng 0 nếu không có cha.
3. Duyệt `ofile[]`: với mỗi fd đang mở, ghi số fd vào `fds` và `fileidx(ofile[fd])` vào `fidx`; `nfd` là số fd tìm được.
4. Nhả `p->lock` và `wait_lock`, rồi `copyout` biến `sproc` vào phần tử thứ `count` của mảng `procs` ở phía user và tăng `count`; nếu `copyout` lỗi thì trả về -1.

Cuối cùng hàm trả về `count`, chính là số process khác `UNUSED`.

4.3.3 Các giải pháp kỹ thuật cốt lõi trong `proccollect`

1. **Giải quyết bài toán tràn Kernel Stack (Stack Overflow):** Cấu trúc `struct sysinfo` theo thiết kế chứa mảng 100 phần tử `sysinfo_file` và 64 phần tử `sysinfo_proc`, với tổng kích thước lên đến $\sim$ 38.5 KB. Trong khi đó, mỗi luồng kernel trong xv6 chỉ được cấp phát duy nhất một trang bộ nhớ ngăn xếp (`KSTACK = PGSIZE = 4KB`). Nếu khai báo một biến cục bộ `struct sysinfo info` trên stack kernel, hệ thống sẽ lập tức bị tràn trang bảo vệ (guard page) và panic ngay lập tức.

   Giải pháp: Khai báo biến cục bộ duy nhất cho **từng phần tử** `struct sysinfo_proc sproc` (164 bytes). Với mỗi tiến trình hợp lệ, dữ liệu được đóng gói vào `sproc` trên stack rồi lập tức sao chép ra không

gian người dùng bằng `copyout()`. Địa chỉ đích được tính toán chính xác bằng macro trình biên dịch `__builtin_offsetof(struct sysinfo, procs) + count * sizeof(struct sysinfo_proc)`.

2. **Thứ tự khóa (Lock Ordering) chống Deadlock:** Trường `p->parent` được bảo vệ bởi spinlock `wait_lock`. Trong xv6, ở các hàm quan trọng như `exit()` và `wait()`, thứ tự khóa luôn được quy định: `wait_lock` **được lấy trước** `p->lock`. Nếu trong `proccollect` ta đảo ngược thứ tự (lấy `p->lock` trước rồi mới lấy `wait_lock`), một tiến trình khác đang thực thi `exit()` (đã giữ `wait_lock` và đang chờ lấy `p->lock`) sẽ gây ra tình trạng khóa chết vòng tròn (circular wait deadlock). Do đó, hàm tuân thủ nghiêm ngặt quy tắc: lấy `wait_lock` trước, sau đó mới lấy `p->lock`.

3. **Thả khóa trước khi thực hiện** `copyout()`**:** Hàm `copyout()` thực hiện truy xuất bảng phân trang và ghi dữ liệu vào không gian bộ nhớ của tiến trình người dùng, vốn là thao tác tiềm ẩn lỗi hoặc gây trễ. Trong xv6, việc giữ spinlock khi truy cập dữ liệu người dùng có thể dẫn đến vi phạm nghiêm trọng (làm tăng thời gian giữ khóa hoặc vi phạm các assertion về ngắt). Vì vậy, dữ liệu của tiến trình được chụp nhanh (snapshot) vào biến cục bộ `sproc`, toàn bộ khóa `p->lock` và `wait_lock` được giải phóng ngay sau đó, rồi mới gọi `copyout()`.

4. **Xử lý tiến trình gốc (**`init`**):** Tiến trình đầu tiên `initproc` không có cha (`p->parent == 0`). Hàm kiểm tra điều kiện con trỏ `p->parent` trước khi truy xuất, gán `ppid = 0` một cách an toàn để tránh dereference con trỏ NULL gây kernel trap.

4.4 Danh sách các file đã sửa đổi

- `kernel/sysinfo.h`**:** Thiết kế và định nghĩa các cấu trúc `struct sysinfo_file`, `struct sysinfo_proc` và `struct sysinfo` theo đúng đặc tả thống nhất của nhóm.
- `kernel/defs.h`**:** Khai báo nguyên mẫu các hàm mới gồm `int filecollect(uint64);`, `int fileidx(struct file*);`, và `int procs_using_file(struct file*, int*);`.
- `kernel/file.c`**:** Bổ sung `#include "sysinfo.h"`, cài đặt hàm `int fileidx(struct file *f)` chuyển đổi con trỏ file sang chỉ số ô trong `ftable.file[]`, và hàm `int filecollect(uint64 addr)` thu thập danh sách open file có `ref > 0`, copyout từng phần tử `struct sysinfo_file` ra buffer người dùng.
- `kernel/proc.c`**:** Cài đặt hàm `int procs_using_file(struct file *f, int *pids)` duyệt mảng `proc[]`, quét danh sách `ofile[]` dưới `p->lock` để thu thập danh sách PID các tiến trình đang mở file `f` (mỗi tiến trình chỉ lưu 1 lần dù có nhiều fd trỏ tới file).
- `kernel/sysproc.c`**:** Bổ sung `#include "sysinfo.h"`, cài đặt hàm xử lý system call `sys_sysinfo()` lấy địa chỉ user từ thanh ghi đối số qua `argaddr()`, gọi `filecollect(addr)` và `proccollect(addr)`, sau đó copyout hai giá trị `nfile` và `nproc` vào đầu cấu trúc. (Trong lúc phát triển, A có dùng tạm một hàm weak stub `proccollect()` để biên dịch độc lập, xem mục 7; stub này đã được xóa khi ghép với phần của C.)

- `user/sysinfotest.c`: Viết lại toàn bộ, vì file có sẵn trong repo sau khi clone được viết cho `struct sysinfo` cũ (chỉ có `freemem` và `nproc`) nên không còn dùng được với struct mới. Chương trình gồm các hàm tra cứu kết quả (tìm process theo pid, tìm file theo `idx`, từ fd ra file), hàm in danh sách file và process thành bảng, các hàm kiểm tra in `PASS`/`FAIL`, và 17 hàm test ứng với 17 kịch bản ở mục 6. File đã được thêm vào `UPROGS` trong `Makefile` (phần của C).

- `kernel/syscall.h`, `kernel/syscall.c`, `user/usys.pl`, `user/user.h`: Đăng ký system call mới `sysinfo`: định nghĩa mã số `SYS_sysinfo` (22) trong `syscall.h`; khai báo ngoại vi `extern uint64 sys_sysinfo(void);` và đăng ký phần tử `[SYS_sysinfo] sys_sysinfo` vào bảng con trỏ hàm `syscalls[]` trong `syscall.c`; thêm chỉ thị `entry("sysinfo");` vào `usys.pl` để Perl script tự động sinh mã stub hợp ngữ (`li a7, SYS_sysinfo; ecall; ret`) vào `usys.S`; khai báo trước cấu trúc `struct sysinfo;` và prototype phía user `int sysinfo(struct sysinfo *);` trong `user.h`.

- `Makefile`: Bổ sung mục tiêu biên dịch `$U/_sysinfotest` vào danh sách chương trình người dùng `UPROGS` trong khối điều kiện `ifeq ($(LAB),syscall)`.

- `kernel/proc.h`, `kernel/proc.c`: Thêm trường `int nsched;` vào cấu trúc `struct proc` (vùng dữ liệu được bảo vệ bởi spinlock `p->lock`); nạp thư viện `#include "sysinfo.h"`; khởi tạo `p->nsched = 0;` trong `allocproc()`; tăng bộ đếm `p->nsched++;` trong `scheduler()` ngay trước khi thực hiện `swtch()`; hiện thực hàm `int proccollect(uint64 addr)` thu thập thông tin các tiến trình khác `UNUSED` và copyout từng phần tử `struct sysinfo_proc` ra không gian người dùng.

- `kernel/defs.h`: Khai báo nguyên mẫu hàm `int proccollect(uint64);` trong nhóm `proc.c` và `int fileidx(struct file*);` trong nhóm `file.c`.

- `kernel/sysinfo.h`: Cập nhật toàn bộ cấu trúc dữ liệu theo đúng đặc tả thống nhất của nhóm (`struct sysinfo_file`, `struct sysinfo_proc`, `struct sysinfo`).

5 Lựa chọn thiết kế và giả định

- **Cơ chế `copyout` từng phần tử thay vì toàn bộ cấu trúc:** Do `struct sysinfo` có kích thước xấp xỉ 38.5 KB, trong khi kernel stack của xv6 chỉ có đúng 1 trang bộ nhớ vật lý 4 KB (`PGSIZE = 4096`), việc khai báo toàn bộ cấu trúc trên stack sẽ gây tràn ngăn xếp (kernel stack overflow) và panic ngay lập tức. Thiết kế lựa chọn giải pháp chỉ khởi tạo từng phần tử đơn lẻ (`struct sysinfo_file` khoảng 280 bytes) trên stack và dùng `copyout()` chép trực tiếp vào đúng vị trí đích trên không gian nhớ của tiến trình người dùng thông qua `__builtin_offsetof`.

- **Bảo mật bộ nhớ kernel bằng `memset`:** Trước khi ghi nhận dữ liệu cho `struct sysinfo_file`, hàm `memset(&sf, 0, sizeof(sf))` được gọi để xóa sạch toàn bộ các byte trên ngăn xếp nhân (bao gồm cả các byte padding giữa các trường). Thao tác này ngăn chặn hoàn toàn nguy cơ rò rỉ dữ liệu nhạy cảm của kernel (kernel information leak) ra không gian người dùng khi thực hiện `copyout()`.

- **Chiến lược Snapshot và Thứ tự Lock:** Để tránh nguy cơ bế tắc (AB-BA deadlock) giữa khóa bảng tập tin toàn cục `ftable.lock` và khóa tiến trình `p->lock`, kernel chỉ giữ `ftable.lock` để sao chép nhanh các trường thông tin cơ bản của file, sau đó giải phóng khóa ngay trước khi chuyển sang bước tra cứu tiến trình `procs_using_file()` và bước gọi `copyout()`.

- **Giả định về tính nhất quán thống kê:** Việc kiểm tra mảng `ofile[]` của tiến trình khác mà không đóng băng toàn bộ hệ thống chấp nhận một cửa sổ thời gian cực nhỏ (race window) nếu tiến trình đó đang đồng thời thực hiện thao tác `close()`. Với mục đích thống kê và giám sát trạng thái hệ điều hành tại một thời điểm, giả định lấy mẫu độc lập tại thời khắc quan sát (snapshot observation) là hoàn toàn phù hợp và tối ưu về hiệu năng.

- **So sánh trước và sau thay vì so với giá trị cố định:** Nhiều giá trị như `ref` của console, pid hay `nsched` phụ thuộc vào `init`, `sh` và thời điểm chạy. Vì vậy mỗi test chụp kết quả `sysinfo` trước và sau thao tác rồi kiểm tra độ chênh lệch, ví dụ `nfile` tăng đúng 5 sau 5 lần `open`. Chỉ các hằng số của hệ thống (`NOFILE`, `NFILE`, `NPROC`, `init` có pid 1) mới được so sánh trực tiếp.

- **Giả định hệ thống không có chương trình khác chạy song song:** `sysinfotest` được chạy từ shell và lúc đó chỉ có `init`, `sh` đang chờ. Kernel thu thập danh sách file và danh sách process ở hai thời điểm khác nhau, nên các kiểm tra đối chiếu chéo giữa hai danh sách chỉ được thực hiện khi mọi process khác đang bị chặn. Khi chờ một process đổi trạng thái, test chụp thêm một lần sau khi thấy trạng thái đó rồi mới đối chiếu, vì lần chụp đầu tiên có thể lấy danh sách file trước lúc process đổi trạng thái (mục 7). Mỗi test tự dọn dẹp (`close`, `wait`, `kill`) và kiểm tra `nfile`, `nproc` đã trở về như ban đầu trước khi sang test sau.

- **`struct sysinfo` khai báo toàn cục:** Struct này khoảng 38 KB trong khi user stack của xv6 chỉ có một trang 4 KB (`USERSTACK = 1`), nên không thể khai báo làm biến cục bộ.

- **Giữ process con đứng yên bằng pipe:** Khi cần chụp kết quả lúc con còn sống, con bị chặn ở `read()` trên một pipe chưa có dữ liệu, cha ghi 1 byte để cho con chạy tiếp và thoát. Cách này cho kết quả chắc chắn hơn so với chờ bằng `sleep`, và con giữ nguyên bảng fd giống hệt cha tại thời điểm chụp. Khi cần chờ một trạng thái cụ thể (`ZOMBIE`, `SLEEPING`), test gọi `sysinfo` liên tục tới khi thấy trạng thái đó, tối đa 100 tick (đo bằng `uptime()`).

- **Không phụ thuộc vào số CPU:** Test 10 chỉ khẳng định các con chạy vòng lặp vô hạn luôn ở `RUNNING` hoặc `RUNNABLE` và có lúc thấy `RUNNABLE`. Số con đang `RUNNING` chỉ được in ra để quan sát, vì khi chạy với 1 CPU thì CPU đó đang chạy chính `sysinfotest`, nên không bao giờ thấy con nào `RUNNING`. Nhóm đã chạy với cả `CPUS=3` và `CPUS=1`, đều đạt.

- **Định nghĩa sự kiện lập lịch (*Scheduling Event*):** Nhóm quy ước một sự kiện lập lịch của một tiến trình là một lần tiến trình đó được `scheduler()` chọn từ trạng thái `RUNNABLE` và thực hiện chuyển ngữ cảnh (`swtch()`) để trao quyền thực thi trên CPU. Việc chỉ tăng `p->nsched++` tại một vị trí duy nhất trong `scheduler()` ngay trước `swtch()` giúp logic tập trung (single point of truth), tránh can thiệp rải

rác vào nhiều hàm nhường CPU khác nhau (`yield`, `sleep`, `exit`) và loại trừ hoàn toàn nguy cơ đếm trùng lặp.

- **Chiến lược sao chép từng phần tử thay vì toàn bộ cấu trúc (*Streaming Copyout*):** Toàn bộ `struct sysinfo` có kích thước ≈ 38.5 KB, trong khi kernel stack chỉ có đúng 1 trang 4 KB (`PGSIZE = 4096`). Để triệt tiêu nguy cơ tràn kernel stack mà không phải cấp phát động phức tạp hay làm cạn kiệt bộ nhớ trang qua `kalloc()`, hàm `proccollect()` chỉ khai báo biến cục bộ cho từng phần tử `struct sysinfo_proc` (164 bytes), đóng gói dữ liệu rồi lập tức ghi vào không gian user bằng `copyout()` thông qua tính offset chính xác bằng `__builtin_offsetof`.
- **Thứ tự khóa (*Lock Ordering*) chống Deadlock:** Để lấy `ppid`, ta cần truy xuất `p->parent->pid` vốn được bảo vệ bởi `wait_lock`. Trong xv6, ở các hàm như `exit()` và `wait()`, `wait_lock` luôn được lấy trước `p->lock`. Do đó, `proccollect()` tuân thủ nghiêm ngặt: lấy `wait_lock` trước, sau đó mới lấy `p->lock`, và giải phóng theo thứ tự ngược lại để ngăn chặn tình trạng khóa chết vòng tròn (circular wait deadlock).
- **Giả định về việc đọc bảng `ofile[]` trong `proccollect`:** Trong xv6, mảng `ofile[NOFILE]` là thuộc tính private của tiến trình. Khi `proccollect()` duyệt qua `ofile[]` của tiến trình khác, thao tác được bảo vệ dưới khóa `p->lock`. Thiết kế chủ động không giữ `ftable.lock` trong khi đang giữ `p->lock` để tránh tranh chấp thứ tự khóa gây deadlock trên hệ thống đa lõi; kết quả đọc là một bản chụp thời điểm (point-in-time snapshot) hoàn toàn chấp nhận được cho mục đích chẩn đoán hệ thống.

6 Kết quả kiểm thử

6.1 Cách kiểm thử

Chương trình `user/sysinfotest.c` gọi `sysinfo` trước và sau mỗi thao tác (`open`, `dup`, `fork`, `exit`, ...) rồi so sánh hai lần chụp. Mỗi kiểm tra in một dòng `[PASS]` hoặc `[FAIL]` kèm giá trị đo được, cuối chương trình in tổng số kiểm tra đạt và không đạt. Chương trình có ba cách chạy trong shell của xv6:

- `sysinfotest`: chạy cả 17 test;
- `sysinfotest 5 8`: chỉ chạy các test được liệt kê;
- `sysinfotest show`: chỉ in danh sách open file và danh sách process tại thời điểm gọi.

Ngoài các kiểm tra riêng của từng kịch bản, nhiều test còn kiểm tra hai tính chất chung của kết quả trả về. Thứ nhất, danh sách file và danh sách process phải khớp nhau: mỗi pid trong danh sách process của một file phải có fd trỏ tới file đó, và ngược lại. Thứ hai, khi không có process nào đang mở hay đóng file, `ref` của mỗi file phải bằng tổng số fd trỏ tới nó. Cuối mỗi test luôn có một kiểm tra `nfile` và `nproc` đã trở về như lúc bắt đầu, để chắc test đã dọn dẹp xong và không ảnh hưởng tới test sau.

Nhóm chạy chương trình trên QEMU với 3 CPU (mặc định của `make qemu`) ba lần liên tiếp và một lần với `make qemu CPUS=1`. Cả bốn lần đều đạt 118/118 kiểm tra và kernel không panic.

6.2 Tổng hợp các kịch bản

#	Kịch bản	Nội dung kiểm tra chính	Số KT
1	Trạng thái ban đầu	Có `init` (pid 1, ppid 0), `sh` đang `SLEEPING` trong `wait`, `sysinfotest` đang `RUNNING`; fd 0, 1, 2 cùng trỏ console (`DEVICE`, `rw`); hai danh sách khớp nhau	13
2	`open` với từng chế độ	`O_RDONLY`, `O_WRONLY`, `O_RDWR` cho `INODE` với quyền `r-`, `-w`, `rw`; mở thư mục cho `INODE` `r-`; mở lại console cho một entry `DEVICE` mới	10
3	`open` một file hai lần	Hai entry khác nhau, mỗi entry `ref = 1`, offset độc lập	6
4	`dup`	Không thêm entry, `ref` tăng lên 2, pid chỉ có một lần, offset dùng chung	6
5	`fork`	`nproc` tăng 1, con có `ppid` đúng và cùng fd với cha, `ref` tăng theo số fd được chép, offset dùng chung, sau `wait` mọi thứ trở lại như cũ	14
6	`pipe`	Hai entry `PIPE`: đầu đọc `r-`, đầu ghi `-w`	4
7	`close`	Đóng một trong hai fd: `ref` giảm còn 1; đóng fd cuối: entry biến mất	7
8	Process `ZOMBIE`	Con đã `exit` nhưng cha chưa `wait`: trạng thái `ZOMBIE`, không còn fd; sau `wait` con biến mất, mã thoát đúng	9
9	Process `SLEEPING`, `kill`	Con chờ `read` trên pipe rỗng và con trong `sleep(1000)` đều `SLEEPING`; con bị `kill` thoát với mã -1	5
10	`RUNNING` / `RUNNABLE`	4 con chạy vòng lặp vô hạn (nhiều hơn số CPU): luôn ở `RUNNING` hoặc `RUNNABLE`, có lúc thấy `RUNNABLE`; `nsched` của mỗi con tăng dần	5
11	Đếm `nsched`	`nsched` của chính chương trình tăng sau `sleep`; con chờ pipe không tăng; con trong `sleep()` vẫn tăng	4
12	Nhiều process con	5 con: `nproc` tăng 5, mọi con có `ppid` đúng; pipe dùng chung có `ref = 6`	6
13	Process mồ côi	Cha của cháu thoát trước: cháu được chuyển cho `init` (`ppid = 1`), `init` thu dọn cháu khi nó thoát	5
14	Giới hạn `NOFILE`	Mở tới khi `open` lỗi: process có đúng 16 fd, từ 0 tới 15	5

#	Kịch bản	Nội dung kiểm tra chính	Số KT
15	Giới hạn `NFILE`	10 process con mở file tới khi lỗi: `nfile` đạt 100; `open` lỗi do `ftable` đầy dù process còn fd trống	6
16	Giới hạn `NPROC`	Fork tới khi lỗi: tạo được 61 con, `nproc` đạt 64	4
17	Con trỏ không hợp lệ	Địa chỉ 0 (trang code chỉ đọc), vượt `MAXVA`, trang `TRAPFRAME`, guard page của stack, buffer tràn qua cuối heap: đều trả về -1; buffer cấp bằng `malloc` hoạt động bình thường	9
		Tổng	**118**

6.3 Một số kết quả tiêu biểu

Các hình dưới đây là output thật của lần chạy với 3 CPU. Dòng `...` là chỗ đã lược bớt.

Trạng thái ban đầu (test 1). Hệ thống chỉ có một open file là console, được 3 process dùng chung với `ref = 9` (mỗi process 3 fd, xem mục 3.3). `sysinfotest` là process duy nhất đang `RUNNING`, vì nó chính là process đang gọi `sysinfo`.

```
== test 1: baseline: init, sh, sysinfotest, console ==
  open files: 1
    idx  type    mode  ref  pids
    0    DEVICE  rw    9    1 2 3
  processes: 3
    pid  ppid  state     nsched  name          fd->idx
    1    0     SLEEPING  24      init          0->0 1->0 2->0
    2    1     SLEEPING  11      sh            0->0 1->0 2->0
    3    2     RUNNING   48      sysinfotest   0->0 1->0 2->0
  [PASS] file list and process list agree with each other
  [PASS] ref of every file = number of fds pointing to it
  ...
  [PASS] state of sh (blocked in wait) = SLEEPING
  [PASS] fd 0, 1, 2 share one struct file (init used dup)
```

Hình 1: Kết quả test 1: danh sách open file và process lúc bắt đầu

`open` với từng chế độ (test 2). Mỗi lần `open` tạo một entry mới với `ref = 1`; quyền đọc/ghi đúng theo chế độ mở. Mở lại `console` cho entry `DEVICE` thứ hai (idx 5), khác với entry idx 0 mà fd 0, 1, 2 đang dùng.

```
== test 2: open() with each mode, a directory and a device ==
  open files: 6
    idx  type    mode  ref  pids
    0    DEVICE  rw    9    1 2 3
    1    INODE   r-    1    3
    2    INODE   -w    1    3
    3    INODE   rw    1    3
    4    INODE   r-    1    3
    5    DEVICE  rw    1    3
```

Hình 2: Kết quả test 2: các entry do `open` tạo ra

`fork` (test 5). Con (pid 4) có `ppid = 3`, đúng các fd của cha và trỏ tới cùng các entry. Con đang `RUNNABLE` với `nsched = 0` vì chưa được scheduler chọn lần nào.

```
== test 5: fork() ==
  processes: 4
    pid  ppid  state     nsched  name          fd->idx
    1    0     SLEEPING  24      init          0->0 1->0 2->0
    2    1     SLEEPING  11      sh            0->0 1->0 2->0
    3    2     RUNNING   50      sysinfotest   0->0 1->0 2->0 3->1 4->2 5->3
    4    3     RUNNABLE  0       sysinfotest   0->0 1->0 2->0 3->1 4->2 5->3
  [PASS] nproc increase = 1
  [PASS] nfile increase (fork creates no struct file) = 0
  ...
  [PASS] child has the same fds, pointing to the same struct file
  [PASS] each file gains one ref per inherited fd, pids has parent and child
  [PASS] console ref increase (fd 0, 1, 2 inherited) = 3
  ...
  [PASS] offset is shared with the child (child read 'a', parent reads 'b')
```

Hình 3: Kết quả test 5: process con ngay sau `fork`

Process `ZOMBIE` (test 8). Con đã gọi `exit(7)` nhưng cha chưa `wait`: ô của con vẫn còn trong bảng process ở trạng thái `ZOMBIE` và không còn fd nào, vì `exit()` đã đóng hết. Sau `wait`, ô được giải phóng và con không còn trong danh sách.

```
== test 8: ZOMBIE until wait() ==
  processes: 4
    pid  ppid  state     nsched  name          fd->idx
    ...
    3    2     RUNNING   54      sysinfotest   0->0 1->0 2->0
    6    3     ZOMBIE    1       sysinfotest
  [PASS] state of exited child before wait() = ZOMBIE
  [PASS] nfd of zombie (exit closed all fds) = 0
  ...
  [PASS] exit status = 7
  [PASS] after wait() the child is no longer listed
```

Hình 4: Kết quả test 8: process con ở trạng thái `ZOMBIE`

`RUNNING`/`RUNNABLE` và `nsched` (test 10, 11). Bốn con chạy vòng lặp vô hạn trên 3 CPU nên luôn có con phải chờ ở `RUNNABLE`. Sau mỗi lần timer ngắt, process đang chạy phải `yield()` rồi được chọn lại, nên `nsched` của các con tăng liên tục. Ngược lại, con chờ trên pipe rỗng không được đánh thức nên `nsched` đứng yên.

```
== test 10: RUNNING / RUNNABLE spinners ==
    pid  ppid  state     nsched  name          fd->idx
    ...
    9    3     RUNNABLE  1       sysinfotest   0->0 1->0 2->0
    10   3     RUNNABLE  1       sysinfotest   0->0 1->0 2->0
    11   3     RUNNING   2       sysinfotest   0->0 1->0 2->0
    12   3     RUNNING   1       sysinfotest   0->0 1->0 2->0
  [PASS] spinning children are always RUNNING or RUNNABLE
  [PASS] with more spinners than CPUs some of them wait as RUNNABLE
         pid 9: nsched 2 -> 5
         pid 10: nsched 1 -> 6
  ...
== test 11: nsched counting ==
  [PASS] nsched of sysinfotest grows after sleep(2): 67 -> 69
  [PASS] nsched of child blocked on a pipe does not change: 1 -> 1
  [PASS] nsched of child in sleep() grows (woken at every tick): 1 -> 5
```

Hình 5: Kết quả test 10 và 11: trạng thái chạy và bộ đếm `nsched`

Giới hạn của hệ thống và con trỏ không hợp lệ (test 13–17). Process mồ côi được chuyển cho `init`. Các bảng đầy (`NOFILE`, `NFILE`, `NPROC`) vẫn được `sysinfo` trả về đầy đủ. Mọi địa chỉ user không hợp lệ đều làm `sysinfo` trả về -1 mà không làm hỏng kernel hay process gọi.

```
== test 13: orphan reparented to init ==
  [PASS] ppid of orphan (reparented to init) = 1
  [PASS] init reaps the orphan after it exits
== test 14: per-process limit NOFILE ==
  [PASS] nfd of sysinfotest = 16
== test 15: system-wide limit NFILE ==
  [PASS] nfile (system-wide file table full) = 100
  [PASS] open() fails because ftable is full, not because of NOFILE
== test 16: process table full (NPROC) ==
  [PASS] nproc (process table full) = 64
  [PASS] children forked until fork() failed = 61
== test 17: bad user pointers ==
  [PASS] sysinfo(0), page 0 is read-only text = -1
  [PASS] sysinfo(0xeaeb0b5b00002f5e), above MAXVA = -1
  [PASS] sysinfo(TRAPFRAME), kernel-only page = -1
  [PASS] sysinfo(stack guard page) = -1
  [PASS] sysinfo(heap end - 16), buffer runs past the heap = -1
  [PASS] sysinfo(buffer from malloc) = 0
  [PASS] sysinfo(&global) still works afterwards = 0

sysinfotest: 118 passed, 0 failed
sysinfotest: OK
```

Hình 6: Kết quả test 13–17 và tổng kết (đã lược bớt một số dòng)

7 Khó khăn và các cách đã thử

1. **Giới hạn kích thước ngăn xếp của xv6 và lỗi tràn stack:**

   - *Hiện tượng / Nguy cơ:* Kích thước tổng cộng của `struct sysinfo` gần 40 KB, vượt xa dung lượng 4 KB của kernel stack. Nếu khai báo biến cục bộ dạng `struct sysinfo info;` trong kernel, ngăn xếp sẽ lập tức bị tràn qua guard page và làm sập hệ thống (page fault panic).
   - *Giải quyết:* Tính toán chính xác địa chỉ ảo của từng phần tử trên trang nhớ người dùng bằng macro `__builtin_offsetof(struct sysinfo, files) + k * sizeof(struct sysinfo_file)`, từ đó khởi tạo từng phần tử nhỏ trên stack rồi `copyout()` từng phần tử một.

2. **Nguy cơ Deadlock tiềm ẩn khi duyệt liên bảng giữa `ftable` và `proc`:**

   - *Phân tích:* Khi thu thập các tiến trình đang mở một file, nếu giữ `ftable.lock` trong lúc gọi hàm duyệt `proc[]` (vốn cần `p->lock`), có thể xảy ra deadlock vòng lặp với các syscall như `sys_open()` hay `sys_close()` vốn đang giữ `p->lock` và đợi `ftable.lock`.
   - *Giải quyết:* Áp dụng mô hình Snapshot: chỉ giữ `ftable.lock` trong thời gian tối thiểu để trích xuất dữ liệu của file ra biến cục bộ, sau đó mở khóa rồi mới tiến hành tra cứu tiến trình qua `procs_using_file()` và gọi `copyout()`.

3. **Trùng lặp PID khi tiến trình dùng `dup()`:**

   - *Phân tích:* Nếu một tiến trình mở một file rồi dùng lệnh `dup()` để nhân bản descriptor, mảng `ofile[]` sẽ chứa nhiều descriptor cùng trỏ tới một `struct file`. Nếu chỉ duyệt đơn thuần, PID của tiến trình đó sẽ bị ghi nhiều lần vào mảng `pids[]`.
   - *Giải quyết:* Sử dụng cờ đánh dấu `found` trong vòng lặp kiểm tra descriptor của mỗi tiến trình; ngay khi tìm thấy descriptor đầu tiên khớp với tập tin, tiến hành dừng duyệt (`break`) và chỉ thêm PID vào danh sách đúng 1 lần.

4. Phát triển song song không phụ thuộc vào thành viên khác:

- *Phân tích:* Phần cài đặt của A cần gọi hàm `proccollect()` do thành viên C phụ trách. Nếu C chưa hoàn thành mã nguồn, trình liên kết (linker) sẽ báo lỗi *undefined reference* và không thể build kiểm thử.
- *Giải quyết:* Khai báo hàm `proccollect()` dưới dạng liên kết yếu (`__attribute__((weak))`) trong `kernel/sysproc.c`. Nhờ đó, mã nguồn của A vẫn biên dịch và kiểm thử độc lập thành công. Khi thành viên C hoàn thành phần cài đặt chính thức trong `kernel/proc.c`, hàm của C sẽ tự động ghi đè lên hàm yếu mà không gây xung đột. Khi ghép code, nhóm đã xóa stub này, bản nộp chỉ còn `proccollect()` trong `kernel/proc.c`.

5. Process con đang `sleep()` luôn bị thấy ở trạng thái `RUNNABLE`:

- *Hiện tượng:* Ở phiên bản đầu, test 9 báo `child blocked in sleep(1000) = RUNNABLE (expected SLEEPING)`, dù chương trình đã chờ tới 100 lần. Con chờ `read()` trên pipe thì vẫn được thấy `SLEEPING` bình thường.
- *Phân tích:* Vòng chờ lúc đó gọi `sysinfo` rồi `sleep(1)`. `sys_sleep` ngủ trên kênh `&ticks`, và mỗi tick `clockintr()` gọi `wakeup(&ticks)`, đánh thức mọi process đang ngủ trên kênh này, gồm cả cha lẫn con. Cha vừa thức dậy là gọi `sysinfo` ngay, đúng lúc con cũng vừa bị đánh thức để kiểm tra lại số tick, nên lần nào con cũng đang `RUNNABLE` hoặc `RUNNING`.
- *Giải quyết:* Vòng chờ gọi `sysinfo` liên tục, không ngủ, và giới hạn thời gian bằng `uptime()`. Từ lỗi này nhóm hiểu thêm: process trong `sleep(n)` thực ra được đưa lên CPU lại ở mỗi tick, nên `nsched` của nó vẫn tăng (test 11: từ 1 lên 5 sau 5 tick). Process chờ trên pipe chỉ được đánh thức khi có dữ liệu nên `nsched` đứng yên.

6. Test 8 thỉnh thoảng thấy process `ZOMBIE` vẫn còn giữ file:

- *Hiện tượng:* Có lần chạy chỉ đạt 117/118, kiểm tra `zombie is not in pids of any file` bị `FAIL`: con đã là `ZOMBIE`, không còn fd nào, nhưng pid của nó vẫn nằm trong danh sách process của console.
- *Phân tích:* Một lần gọi `sysinfo` lấy danh sách file trước, danh sách process sau. Lúc lấy danh sách file, con vẫn đang chạy trong `exit()` và còn giữ fd 0, 1, 2; tới lúc lấy danh sách process thì con đã đóng hết fd và thành `ZOMBIE`. Vòng chờ dừng đúng ở lần chụp này, nên hai danh sách trong cùng một lần chụp không khớp nhau.
- *Giải quyết:* Sau khi thấy con là `ZOMBIE`, test gọi `sysinfo` thêm một lần rồi mới kiểm tra. `ZOMBIE` không thay đổi gì cho tới khi cha gọi `wait`, nên lần chụp sau chắc chắn nhất quán. Đây là lỗi của test, không phải của kernel.

7. Máy không có sẵn trình biên dịch RISC-V và QEMU:

- *Giải quyết:* Nhóm dựng một Docker image Ubuntu 24.04 có `gcc-riscv64-linux-gnu` và `qemu-system-misc`, mount thư mục mã nguồn vào để `make qemu`. Nhóm dùng thêm `expect` để tự

động gõ `sysinfotest` vào shell của xv6, nhờ vậy chạy lại toàn bộ test nhiều lần (3 lần với 3 CPU, 1 lần với `CPUS=1`) để chắc kết quả không phụ thuộc thời điểm chạy.

8. **Tràn Kernel Stack 4KB do kích thước lớn của `struct sysinfo`:**

   - *Hiện tượng:* Cấu trúc `struct sysinfo` theo yêu cầu có kích thước ≈ 38.5 KB (100 files và 64 procs). Ngăn xếp kernel của mỗi tiến trình trong xv6 chỉ có đúng 4 KB kèm một guard page không hợp lệ bao quanh. Việc khai báo trực tiếp `struct sysinfo` trên kernel stack khiến con trỏ `sp` tràn ra ngoài vùng hợp lệ, gây panic kernel trap ngay lập tức.
   - *Phương án tĩnh (`static`):* Đặt biến tĩnh toàn cục trong BSS giúp tránh tràn stack nhưng gây xung đột nghiêm trọng (race condition) khi nhiều CPU cùng gọi `sysinfo()`, đòi hỏi khóa đồng bộ lớn gây nghẽn hiệu năng.
   - *Phương án cấp phát `kalloc()`:* Hàm `kalloc()` của xv6 chỉ cấp phát từng trang đơn lẻ (4096 bytes), không hỗ trợ cấp phát nhiều trang liên tục (≈ 10 trang liên tiếp) nếu không sửa đổi sâu bộ quản lý trang.
   - *Giải pháp thành công:* Áp dụng chiến lược Streaming Copyout: chỉ dùng biến cục bộ cho từng phần tử `struct sysinfo_proc sproc` (164 bytes). Mỗi tiến trình hợp lệ sau khi thu thập được lập tức sao chép ra user space bằng `copyout()` tại offset `__builtin_offsetof(struct sysinfo, procs) + count * sizeof(struct sysinfo_proc)`, an toàn tuyệt đối về bộ nhớ.

9. **Nguy cơ Deadlock vòng tròn do đảo thứ tự lấy khóa (*Lock Ordering*):**

   - *Hiện tượng:* Để thu thập `ppid`, ta cần truy xuất trường `p->parent->pid` được bảo vệ bởi khóa toàn cục `wait_lock`. Nếu lấy `p->lock` trước rồi mới lấy `wait_lock`, trật tự này ngược hoàn toàn với hàm `exit()` (trong đó `wait_lock` được lấy trước `p->lock`).
   - *Giải quyết:* Trên hệ thống 2 CPU trở lên, nếu một tiến trình con đang `exit()` đồng thời với một tiến trình khác gọi `sysinfo()`, khóa chết vòng tròn (deadlock) chắc chắn sẽ xảy ra. Do đó, `proccollect()` bắt buộc tuân thủ: luôn gọi `acquire(&wait_lock)` trước, sau đó mới gọi `acquire(&p->lock)`, và giải phóng ngược lại trước khi copyout.

10. **Vi phạm quy tắc khi gọi `copyout()` trong khi đang giữ spinlock:**

    - *Hiện tượng:* Ban đầu có thể có xu hướng giữ `p->lock` xuyên suốt quá trình gọi `copyout()` để bảo đảm không có CPU nào can thiệp tiến trình. Tuy nhiên `copyout()` tương tác trực tiếp với bảng phân trang user, có thể gây trễ hoặc xử lý ngoại lệ trang. Việc giữ spinlock trong thao tác này làm tê liệt khả năng nhận ngắt và tăng độ trễ toàn hệ thống.
    - *Giải quyết:* Tách biệt chụp trạng thái và ghi dữ liệu: toàn bộ thông tin tiến trình được sao chép vào biến `sproc` trên stack trong phạm vi bảo vệ của spinlock; giải phóng ngay cả hai khóa `p->lock` và `wait_lock`, sau đó mới thực hiện `copyout()`.

8 Những khái niệm chưa hiểu rõ

1. **Tính nhất quán tuyến tính hóa (Linearizability) so với Snapshot Consistency:** Trong một hệ điều hành thực tế chạy trên nhiều lõi CPU với tần suất mở/đóng tập tin cực lớn, việc lấy một "bức ảnh" hoàn toàn nhất quán tại cùng một khoảnh khắc (point-in-time consistency) trên toàn hệ thống mà không phải tạm dừng mọi lõi CPU (stop-the-world) là một bài toán rất phức tạp. Nhóm mong muốn tìm hiểu sâu hơn về các cơ chế Read-Copy-Update (RCU) hoặc lock-free data structures được áp dụng trong nhân Linux hiện đại để giải quyết bài toán này.

2. **Lấy một bản chụp nhất quán của toàn hệ thống:** `sysinfo` thu thập danh sách file rồi mới tới danh sách process, và đọc `ofile[]` của các process khác trong lúc chúng vẫn có thể đang mở hay đóng file. Vì vậy kết quả chỉ chắc chắn khớp nhau khi hệ thống đang yên. Nhóm chưa rõ làm sao để kernel trả về một bản chụp nhất quán mà không phải giữ cùng lúc `ftable.lock` và `p->lock` của mọi process, khi xv6 không quy định thứ tự giữa hai loại khóa này.

3. **Thời điểm `kill()` thật sự có hiệu lực:** Ở test 10, `kill()` chỉ đặt `p->killed` (và đánh thức process nếu nó đang `SLEEPING`); process chỉ thật sự gọi `exit()` ở lần trap tiếp theo, khi `usertrap()` kiểm tra cờ này. Nhóm chưa rõ với một process đang `RUNNING` trên CPU khác thì phải chờ tối đa bao lâu, và nếu process đang ở giữa một system call dài (ví dụ đang chờ đọc đĩa) thì nó có bị dừng ngay hay phải chờ system call đó xong.

4. **Cơ chế chuyển giao quyền sở hữu khóa bất đối xứng xuyên luồng thực thi (*Cross-Thread Lock Handover*):** Trong mô hình đồng thời thông thường, luồng nào lấy khóa thì luồng đó phải tự giải phóng. Trong xv6, khóa `p->lock` được lấy bởi luồng `scheduler()` trên một CPU, nhưng lại được giải phóng bởi chính luồng của tiến trình `p` sau khi chuyển ngữ cảnh (`swtch()`), và ngược lại khi nhường CPU. Mặc dù hiểu mục đích trực tiếp là ngăn chặn CPU khác chọn chạy tiến trình khi ngữ cảnh chưa nạp xong, nhưng cách thức trạng thái theo dõi khóa (`cpu->noff` và `cpu->intena`) được chuyển đổi chính xác qua lệnh `swtch()` giữa hai kernel stack khác nhau vẫn là một cơ chế rất phức tạp ở tầng hợp ngữ RISC-V. Nhóm mong muốn tìm hiểu thêm cách các OS lớn như Linux xử lý chuyển ngữ cảnh bằng kỹ thuật RCU (Read-Copy Update) hoặc cấu trúc dữ liệu không khóa (lock-free).

5. **Tính an toàn đồng thời khi chụp ảnh bảng File Descriptor (`ofile[]`) từ tiến trình khác:** Mảng `ofile[]` là thuộc tính private của tiến trình và không cần khóa khi tiến trình tự `open()` hoặc `close()`. Khi `proccollect()` duyệt qua `ofile[]` của một tiến trình đang `RUNNING` trên CPU khác, nếu tiến trình đó đồng thời gọi `close(fd)`, liệu con trỏ `struct file *f` có thể bị giải phóng về `ftable` ngay trước khi hàm `fileidx(f)` kịp truy cập hay không. Nhóm chưa rõ có cần bổ sung khóa bảo vệ riêng cho bảng descriptor (như `files_struct->file_lock` trong Linux kernel) để đảm bảo tính nhất quán tuyệt đối giữa các CPU hay không.

Tài liệu tham khảo

[1] R. Cox, F. Kaashoek, and R. Morris, "xv6: a simple, Unix-like teaching operating system." https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf, 2024. Bản ngày 31/08/2024; chương 1 (file descriptor, fork), chương 7 (lập lịch), chương 8 (hệ thống file).

[2] "Operating system – Project 1: File & Process Management." Đề bài đồ án môn Hệ điều hành, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM, 2026.